

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

WEEK-1

TASK-1

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Aim:

To develop a C program that demonstrates how the Linux Operating System executes a command entered by the user by creating a child process using `fork()`, executing the command using `exec()`, allowing the parent process to wait using `wait()`, and displaying the Process IDs (PIDs) of both parent and child processes

Software Requirements:

- Ubuntu 24.04 LTS (WSL2)
- GCC Compiler
- Linux Terminal

Procedure:

1. Open the Ubuntu terminal.
2. Create a C source file named `process.c`.
3. Write the program using `fork()`, `execlp()` and `wait()`.
4. Compile the program using the GCC compiler.
5. Execute the program.

6. Enter a Linux command such as pwd, ls, whoami, or date.
7. Observe the Parent Process ID and Child Process ID displayed on the terminal.
8. Verify that the parent process waits until the child process completes execution.

C Code:

```
GNU nano 8.7.1
#include <string.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {

        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        printf("Command execution failed.\n");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

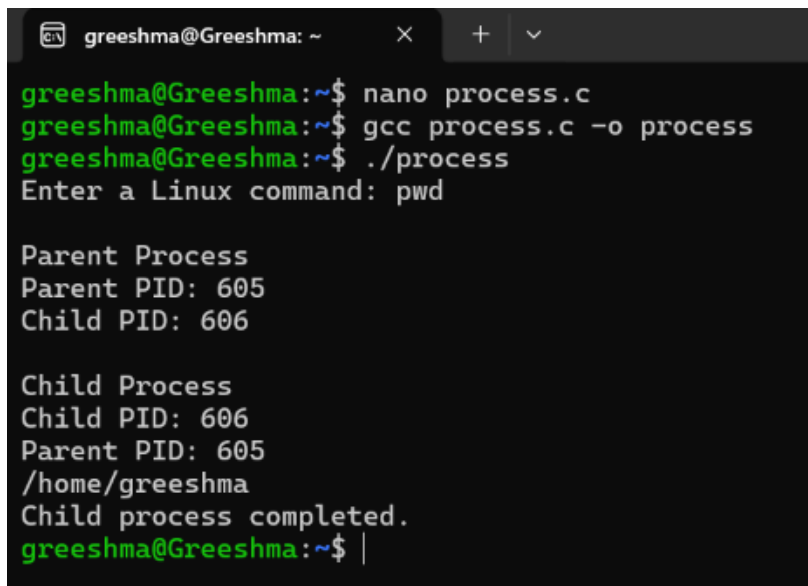
For compilation:

gcc process.c -o process

Execution

./process

Output Screenshot

A terminal window with a dark background and light green text. The window title bar shows 'greeshma@Greeshma: ~' and standard window controls. The terminal shows the following sequence of commands and output:

```
greeshma@Greeshma:~$ nano process.c
greeshma@Greeshma:~$ gcc process.c -o process
greeshma@Greeshma:~$ ./process
Enter a Linux command: pwd

Parent Process
Parent PID: 605
Child PID: 606

Child Process
Child PID: 606
Parent PID: 605
/home/greeshma
Child process completed.
greeshma@Greeshma:~$ |
```

Conclusion

The program successfully demonstrates process creation using `fork()`, command execution using `execlp()`, synchronization using `wait()`, and displays the Process IDs of both parent and child processes in the Linux operating system.